

Práctica Nro. 4

Esquema



APPOINTMENTS (patient_id, patient_name, patient_address, patient_city, primary_phone, secondary_phone, doctor_id, doctor_name, doctor_address, doctor_city, doctor_speciality, appointment_date, appointment_duration, observations, payment_card, contact_phone, medication_name)

Clave Candidata (y Primaria) → (patient_id, doctor_id, appointment_date, medication_name)

Dependencias Funcionales

- **patient_id** → patient_name, patient_address, patient_city, primary_phone, secondary_phone (DF1)
- **doctor_id** → doctor_name, doctor_address, doctor_city, doctor_speciality (DF2)
- **patient_id, appointment_date** → appointment_duration, contact_phone, observations, payment_card (DF3)

Dependencias Multivaluadas

- **patient_id, appointment_date** →> doctor_id (DM1)
- **patient_id, appointment_date** →> medication_name (DM2)

Particiones

- PATIENT (patient_id, patient_name, patient_address, patient_city, primary_phone, secondary_phone)
- DOCTOR (doctor_id, doctor_name, doctor_address, doctor_city, doctor_speciality)

- APPOINTMENT (patient_id, appointment_date, appointment_duration, contact_phone, observations, payment_card)
- MEDICAL_REVIEW (patient_id, appointment_date, doctor_id)
- PRESCRIBED_MEDICATION (patient_id, appointment_date, medication_name)

1. Crea un usuario para las bases de datos usando el nombre 'appointments_user'. Asigne a estos todos los permisos sobre sus respectivas tablas.

a. Cree un usuario sólo con permisos para realizar consultas de selección, es decir que no puedan realizar cambios en la base. Use el nombre 'appointments_select'.

- CREATE USER 'appointments_select'@'localhost' IDENTIFIED WITH CACHING_SHA2_PASSWORD BY 'mariano';
- GRANT SELECT ON appointments.* TO 'appointments_select'@'localhost';

[Corregido]

b. Cree un usuario que pueda realizar consultas de selección, inserción, actualización y eliminación a nivel de filas, pero que no puedan modificar el esquema. Use el nombre 'appointments_update'.

- CREATE USER 'appointments_update'@'localhost' IDENTIFIED WITH CACHING_SHA2_PASSWORD BY 'mariano';

[Corregido]

- GRANT SELECT, INSERT, UPDATE, DELETE ON appointments.* TO 'appointments_update'@'localhost';

c. Cree un usuario que tenga los permisos de los anteriores, pero que además pueda modificar el esquema de la base de datos. Use el nombre 'appointments_schema'.

- CREATE USER 'appointments_schema'@'localhost' IDENTIFIED WITH CACHING_SHA2_PASSWORD BY 'mariano';

- GRANT SELECT, INSERT, UPDATE, ALTER, DELETE ON appointments.* TO 'appointments_schema'@'localhost';

[Corregido]

2. Hallar aquellos pacientes que para todas sus consultas médicas siempre hayan dejado su número de teléfono primario (nunca el teléfono secundario).

```
SELECT p.patient_id
FROM appointments.patient as p INNER JOIN appointments.appointment as a ON
(p.patient_id = a.patient_id)
WHERE p.secondary_phone NOT IN (SELECT contact_phone FROM
appointments.appointment);
```

[Corregido]

3. Crear una vista llamada 'doctors_per_patients' que muestre los id de los pacientes y los id de doctores de la ciudad donde vive el paciente.

```
CREATE VIEW appointments.doctors_per_patient AS
    SELECT p.patient_id, d.doctor_id
    FROM appointments.patient AS p JOIN appointments.doctor AS d
    WHERE p.patient_city = d.doctor_city;
```

[Corregido]

4. Utiliza la vista generada en el ejercicio anterior para resolver las siguientes consultas:

a. Obtener la cantidad de doctores por cada paciente que tiene disponible en su ciudad

```
SELECT doctor_id, count(*)
FROM appointments.doctors_per_patient
GROUP BY doctor_id
```

[Corregido]

b. Obtener los nombres de los pacientes sin doctores en su ciudad

```
SELECT p.patient_name
FROM appointments.patient p
WHERE p.patient_id NOT IN ( SELECT patient_id
                           FROM appointments.doctors_per_patient )
```

[Corregido]

c. Obtener los doctores que comparten ciudad con más de cinco pacientes

```
SELECT doctor_id
FROM appointments.doctors_per_patient
GROUP BY doctor_id
HAVING count(DISTINCT patient_id) > 4
```

[Corregido]

5. Escribe y ejecute la sentencia correspondiente para crear la siguiente tabla:



APPOINTMENTS_PER_PATIENT

idApP: int(11) PK AI
id_patient: int(11)
count_appointments: int(11)
last_update: datetime
user: varchar(16)

```
CREATE TABLE appointments.APPOINTMENTS_PER_PATIENT (
idApP integer auto_increment primary key,
id_patient integer not null,
count_appointments integer not null,
last_update datetime not null,
user varchar(16) not null
);
```

6. Crear un Stored Procedure que realice los siguientes pasos dentro de una transacción:

a. Realizar la siguiente consulta: cada patient (identificado por id_patient), calcule la cantidad de appointments que tiene registradas. Registrar la fecha en la que se realiza esta carga y además del usuario con el se realiza.

```
CREATE DEFINER='root'@'localhost' PROCEDURE `calcular_appointments`()
BEGIN
    DECLARE done INT DEFAULT FALSE;
    DECLARE contador INT;
    DECLARE id_pat INT;
    DECLARE curs CURSOR FOR
        SELECT a.patient_id, count(a.appointment_date)
        FROM patient p
        LEFT JOIN appointment a ON p.patient_id = a.patient_id
        GROUP BY p.patient_id;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = TRUE;
    START TRANSACTION;
    OPEN curs;
    read_loop: LOOP
        FETCH curs INTO id_pat, contador;
        IF done THEN
            LEAVE read_loop;
        END IF;
        INSERT INTO appointments_per_patient (id_patient, count_appointments, last_update, user)
            VALUES(id_pat, contador, NOW(), USER());
    END LOOP;
    CLOSE curs;
    COMMIT;
END
```

```
CALL calcular_appointments();  
SELECT * FROM appointments_per_patient
```

7. Indique si las siguientes afirmaciones sobre triggers son verdaderas o falsas. Justifique las falsas.

a. Un trigger se ejecuta únicamente cuando se inserta una fila en una tabla.

FALSO. Según las filminas de la teoría "[el trigger] se asocia con una tabla y se activa cuando ocurre en ésta un evento en particular" [[clase 8](#), p. 28]

b. Un trigger puede ejecutarse antes o después de la operación, esto es definido automáticamente según el tipo de la operación (UPDATE, INSERT o DELETE)

FALSO. No hay relación alguna entre la keyword que indica cuando se va a ejecutar la operación, y el tipo de operación mismo.

c. Todo trigger debe asociarse a una tabla en concreto.

VERDADERO. Por definición del Trigger en el manual de MySQL.

d. NEW y OLD son palabras clave que permiten acceder a los valores de las filas afectadas y se pueden usar ambos independientemente de la operación utilizada.

FALSO. Con la keyword OLD y NEW solo se pueden hacer ciertas operaciones, que no son necesariamente las mismas entre las dos. Por ejemplo, con OLD no se puede hacer INSERT, y con NEW un DELETE [[clase 8](#), pag. 34]

e. FOR EACH ROW en un trigger se usa para indicar que el trigger se ejecutará una vez por cada fila afectada por la operación.

VERDADERO. Según [Elmasri](#) "Es importante observar el efecto de la cláusula **FOR EACH ROW** opcional, que significa que la regla se activa separadamente para cada tupla. Es lo que se conoce como **trigger a nivel de fila**" [p. 710]

8. Crear un Trigger de modo que al insertar un dato en la tabla Appointment, se actualice la cantidad de appointments del paciente, la fecha de actualización y el usuario responsable de la misma (actualiza la tabla APPOINTMENTS PER PATIENT).

```
CREATE
  TRIGGER trig_app
  AFTER INSERT
  ON appointment FOR EACH ROW
  UPDATE appointments_per_patient app
    SET count_appointments = count_appointments + 1,
        last_update = NOW(),
        user = USER()
    WHERE app.patient_id = NEW.patient_id

INSERT INTO appointments_per_patient
(idApP, id_patient, count_appointments, last_update, user)
VALUES (1, 10000036, 10, '2025-10-21 10:00:00', 'root')
```

[Corregido]

9. Crear un stored procedure que sirva para agregar un appointment, junto el registro de un doctor que lo atendió (medical_review) y un medicamento que se le recetó (prescribed_medication), dentro de una sola transacción. El stored procedure debe recibir los siguientes parámetros: patient_id, doctor_id, appointment_duration, contact_phone, appointment_address, medication_name. El appointment_date será la fecha actual. Los atributos restantes deben ser obtenidos de la tabla Patient (o dejarse en NULL).

```
DELIMITER //
CREATE PROCEDURE agregar_appointment (IN patient_id INTEGER, IN doctor_id INTEGER, IN appointment_duration INTEGER, IN contact_phone VARCHAR(60), IN medication_name VARCHAR(60))
BEGIN
```

```

        DECLARE fecha DATETIME;
        DECLARE observaciones VARCHAR(255);
        DECLARE pago_tarjeta VARCHAR(255);
        SET fecha = NOW();

        SELECT observations, payment_card INTO observaciones, pago_tarjeta
        FROM appointment a
        WHERE a.patient_id = patient_id AND a.appointment_date = NOW();

        INSERT INTO appointment (patient_id, appointment_date, appointment_d
uration, contact_phone, observations, payment_card)
            VALUES (i_patient_id, NOW(), i_appointment_duration, i_contact_pho
ne, observaciones, pago_tarjeta);
        INSERT INTO medical_review (patient_id, appointment_date, doctor_id)
            VALUES (i_patient_id, NOW(), i_doctor_id);
        INSERT INTO prescribed_medication (patient_id, appointment_date, medic
ation_name)
            VALUES (i_patient_id, NOW(), i_medication_name);
    END //
DELIMITER ;

```

[[Consultar](#)]

10. Ejecutar el stored procedure del punto 9 con los siguientes datos:



patient_id: 10004427
doctor_id: 1003
appointment_duration: 30
contact_phone: +54 15 2913 9963
appointment_address: 'Hospital Italiano'
medication_name: 'Paracetamol'

CALL agregar_appointment(10004427, 1003, 30, '+54 15 2913 9963', 'Paracetamol')

[Consultar]

11. Considerando la siguiente consulta:



```
SELECT COUNT(a.patient_id)
FROM appointment a, patient p, doctor d, medical_review mr
WHERE a.patient_id = p.patient_id
  AND a.patient_id = mr.patient_id
  AND a.appointment_date = mr.appointment_date
  AND mr.doctor_id = d.doctor_id
  AND d.doctor_specialty = 'Cardiology'
  AND p.patient_city = 'Rosario'
```

que es lo mismo que hacer

```
EXPLAIN SELECT COUNT(a.patient_id)
FROM doctor d
INNER JOIN medical_review mr ON d.doctor_id = mr.doctor_id
INNER JOIN patient p ON mr.patient_id = p.patient_id
INNER JOIN appointment a ON p.patient_id = a.patient_id AND
a.appointment_date = mr.appointment_date
WHERE d.doctor_specialty = 'Cardiology'
AND p.patient_city = 'Rosario';
```

Analice su plan de ejecución mediante el uso de la sentencia EXPLAIN.

- ¿Qué atributos del plan de ejecución encuentra relevantes para evaluar la performance de la consulta?**
El de filtrado (**filtered**), el de filas (**rows**), ¿y quizás **ref**?
- Observe en particular el atributo type ¿cómo se están aplicando los JOIN entre las tablas involucradas?**

En las tablas de appointment y doctor se está usando el **eq_ref**, que se usa cuando todas las partes de un índice se utilizan por el join, y el índice es una PK o un atributo UNIQUE NOT NULL, la tabla de medical_review utiliza **ref**, que se usa cuando el join no puede seleccionar una sola fila en base al valor de la clave, y por último, se utiliza ALL para la tabla de pacientes, en la que se hace un escaneo completo de la tabla por cada combinación de filas de las tablas pasadas, tiene la peor performance en general.

c. **Según lo que observó en los puntos anteriores, ¿qué mejoras se pueden realizar para optimizar la consulta?**

Podríamos hacer que se lean más rápido las filas de la tabla patient que cumplan con la condición de tener una patient_city llamada Rosario, y las de la tabla doctor que cumplan con la condición de tener una doctor_specialty que sea Cardiology.

"MySQL utiliza los índices para estas operaciones: Para encontrar filas que compartan una clausura WHERE coincidente con frecuencia" [[Manual de MySQL](#)]

d. **Aplice las mejoras propuestas y vuelva a analizar el plan de ejecución. ¿Qué cambios observa?**

```
CREATE INDEX idx_patient ON patient (patient_city);
CREATE INDEX idx_doctor ON doctor (doctor_specialty);

EXPLAIN SELECT COUNT(a.patient_id)
  FROM appointment a, patient p, doctor d, medical_review mr
 WHERE d.doctor_specialty = 'Cardiology'
       AND mr.patient_id = p.patient_id
       AND mr.doctor_id = d.doctor_id
       AND mr.appointment_date = a.appointment_date
       AND p.patient_city = 'Rosario'
       AND p.patient_id = a.patient_id;
```

[[Consultar](#)]